

AI-Native CRM — AI Guide

Use cases, configuration, and how to apply AI for every login and every workflow

Audience: administrators, AI governors, and every end user.

What this guide covers

1. **How the AI works** — the one governed gateway every AI feature runs through.
2. **Use cases & applications** — the full catalogue of AI actions, by module.
3. **How to configure AI** — tenant settings, providers, prompts, agents, and knowledge.
4. **How to apply AI for each login** — which AI each persona uses, and how to reach it.
5. **How to apply AI in workflows** — AI triggers and AI actions inside automations.
6. **Governance & safety** — review, redaction, logging, grounding, and escalation.

One principle underpins everything: AI here is **governed, not magical**. Every call goes through a single gateway, draws its prompt from a managed registry (never hardcoded), checks your permissions, is logged, and — when the output is sensitive — **waits for a human to approve it** before it can be used. Answers that draw on your knowledge base **cite their sources** and **escalate instead of guessing** when they can't find a confident, approved answer.

Part 1 — How AI works here

Every AI capability is composed from five governed building blocks:

Block	What it is	Where you manage it
AI Gateway	The single entry point for all model calls — applies provider/model choice, rate limits, redaction, and logging.	<code>/admin</code> AI settings · <code>GET/PATCH</code> <code>/ai/settings</code>
Prompt Registry	Versioned, approvable prompt templates referenced by <code>templateKey</code> . Prompts are never hardcoded.	<code>/ai-prompts</code> · <code>/ai/prompts</code>
Agent Registry	Named agents with declared tools, data scope, and human-approval rules.	<code>/ai-agents</code> · <code>/ai/agents</code>
AI Actions	Predefined, per-module AI tasks (summaries, drafts, recommendations) with a <code>permitted</code> flag and review gate.	<code>/ai-actions</code> · <code>/ai/actions</code>
RAG / Knowledge	The approved knowledge that <i>grounds</i> answers, with retrieval you can inspect.	<code>/knowledge/*</code> · <code>/ai/knowledge/*</code> , <code>/ai/rag/retrieve</code>

The lifecycle of any AI request: `caller` → `permission check` → `Prompt Registry (resolve template)` → `AI Gateway (provider, rate limit, redaction)` → `execute` → `log (ai_usage_logs)` → `if sensitive: review queue` → `approved output usable` .

Live execution note. The gateway and all governance are live; **live model output is deferred** until production provider credentials/execution backends are enabled — until then actions return governed placeholder output so you can build, permission, and review end to end safely.

Part 2 — AI use cases & applications (by module)

Every action below runs through the gateway, checks a module permission, and logs its run. Actions marked **(review)** are **sensitive** — their output is held as `pending_review` until a human approves it.

Module	AI actions
Leads	Lead summary · Lead score explanation · Follow-up email draft (review) · Lead qualification recommendation (review)
Accounts	Account brief · Relationship summary · Health indicator explanation
Opportunities	Opportunity summary · Deal risk analysis · Next-best-action (review) · Proposal draft outline (review)
Campaigns	Campaign plan generator (review) · Email copy generator (review) · Audience suggestion (review) · Campaign performance summary
Social	Caption generator (review) · Hashtag suggestion · Comment sentiment summary
Support	Ticket summary · Suggested response (review) · Knowledge article recommendation · Escalation summary
Customer Success	Onboarding plan generator (review) · Customer health summary · Churn risk explanation · Adoption recommendation (review) · QBR/EBR outline (review) · Renewal strategy suggestion (review)
Training	Lesson summary · Quiz generator · Learning path suggestion
Partners	Partner performance summary · Partner action plan (review) · Inactivity alert explanation
Resellers	Reseller performance summary · Reseller action plan (review) · Inactivity alert explanation

Plus three cross-cutting experiences:

- **AI Assistant / Ask AI** (`/ai-assistant` , `/ask-ai` , `/ai-help`) — a governed assistant for summaries, drafts, and grounded Q&A wherever you have `ai.use_ai`.
- **Customer-Query Bot** (`/portal` Ask AI; reviewed at `/customer-query`) — answers external customers **only** from approved knowledge, with citations, and escalates (often raising a ticket) when unsure.
- **RAG retrieval** (`/knowledge/rag-console`) — grounding for the above; preview exactly what the AI would retrieve for a query, with permission filtering.

Request types the gateway understands: `completion` , `summary` , `draft` , `classification` , `extraction` , `recommendation` (plus domain types: sales, presales, partners, support, customer_success, training, marketing, general).

Part 3 — How to configure AI

3.1 Tenant AI settings

The master controls for the whole tenant. **UI:** Administration → AI settings. **API:** `GET /ai/settings` , `PATCH /ai/settings` (needs `ai.configure` / `ai.manage_ai`).

Setting	What it does
<code>isEnabled</code>	Master on/off for all AI in the tenant.
<code>defaultProvider</code>	<code>openai</code> · <code>anthropic</code> · <code>azure_openai</code> · <code>local</code> .
<code>defaultModel</code>	e.g. <code>claude-opus-4-8</code> .
<code>rateLimitPerMinute</code>	Throttle to control cost/abuse.
<code>allowUserOverrides</code>	Whether callers may pick a different provider/model per request.
<code>redactionEnabled</code>	Strip sensitive data from prompts/outputs before they leave the gateway.
<code>loggingEnabled</code>	Record every request/response to <code>ai_usage_logs</code> .

```
PATCH /api/v1/ai/settings      # ai.configure
{ "isEnabled": true, "defaultProvider":
"anthropic", "defaultModel": "claude-opus-4-8",
  "rateLimitPerMinute": 60, "redactionEnabled":
true, "loggingEnabled": true,
  "allowUserOverrides": false }
GET /api/v1/ai/providers      # providers
available to this tenant
GET /api/v1/ai/templates      # prompt templates
the gateway can resolve
GET /api/v1/ai/logs    /ai/usage # request logs and
usage (ai.view / view_dashboard / manage_ai)
```

Infrastructure-level (env, set by DevOps):

`AI_GATEWAY_ENABLED` , `AI_DEFAULT_PROVIDER` ,
`AI_DEFAULT_MODEL` , `AI_RATE_LIMIT_PER_MINUTE` , provider keys
(`AI_OPENAI_API_KEY` , `AI_ANTHROPIC_API_KEY` ,
`AI_AZURE_OPENAI_API_KEY` / `ENDPOINT` , `AI_LOCAL_ENDPOINT`),
`AI_EMBEDDING_MODEL` , `AI_VECTOR_BACKEND` ,
`AI_LOG_RETENTION_DAYS` . Adding real provider keys is what flips
placeholder output into live model execution.

3.2 Prompt Registry — manage the prompts behind the AI

Prompts are versioned and approvable, so what the AI says is reviewable and controlled. **UI:** `/ai-prompts` . **API:**

`/ai/prompts` .

```
POST /api/v1/ai/prompts
# create (ai.create / manage_ai)
{ "promptKey": "leads.followup_email", "name":
"Lead follow-up email",
  "module": "leads", "promptRole": "system",
"content": "...with {{variables}}...",
  "guardrails": { ... } }
POST /api/v1/ai/prompts/:id/versions
# new version (edit)
POST /api/v1/ai/prompts/:id/approval
# approve/reject (ai.approve)
POST
/api/v1/ai/prompts/:id/versions/:version/activate
# activate a version
```

Callers and workflows reference a prompt by its `templateKey` — change the template centrally and every caller updates.

3.3 Agent Registry — define what an agent may do

An agent bundles a purpose with **explicit guardrails**: which tools it may call, who may invoke it, what data it can see, and whether it needs human approval. **UI:** `/ai-agents` . **API:**

`/ai/agents` .

Field	Meaning
<code>allowedTools</code>	The tools/functions the agent may use.
<code>allowedRoles</code>	Which roles may invoke it.
<code>dataAccessScope</code>	<code>own</code> · <code>team</code> · <code>module</code> · <code>tenant</code> — how widely it can read data.
<code>requiresHumanApproval</code>	Force a human gate on its outputs.
<code>status</code>	<code>draft</code> · <code>active</code> · <code>inactive</code> .
<code>loggingEnabled</code> / <code>escalationRules</code>	Logging and when to escalate to a human.

```
POST /api/v1/ai/agents          # ai.create /
manage_ai
{ "agentKey": "support.triage", "name": "Support
triage agent", "module": "support",
  "allowedTools": ["search_knowledge","summarize"],
"dataAccessScope": "module",
  "requiresHumanApproval": true, "status": "active"
}
```

3.4 Knowledge / RAG — ground the AI in *your* content

Grounded answers only use knowledge that is **approved +**

published. UI: `/knowledge` , `/knowledge/upload` ,
`/knowledge/articles` , `/knowledge/rag-console` . API:
`/ai/knowledge/*` , `/ai/rag/retrieve` .

1. Add a **source** and **upload documents** → process them (chunking) for retrieval.
2. Author **articles**; move them through **approved** → **published** (only then are they used by AI).
3. Use the **RAG console** (`POST /ai/rag/retrieve`) to preview what the AI retrieves and confirm permission filtering.
4. Watch **knowledge gaps** (`/ai/knowledge/gaps`) — recurring unanswered questions to write articles for.

Part 4 — How to apply AI for each login (persona)

Two ways any permitted user reaches AI: the **Ask AI** assistant (top-level or in-context), and **AI Actions** inside a module record (e.g. "Summarize", "Draft follow-up"). What each login sees depends on its `*.use_ai` / `ai.use_ai` permissions.

Login (persona)	AI they apply	
SDR (sales.development.representative)	Lead summary, score explanation, follow-up draft (review)	C A r
Inside / Account Sales (sales.executive , sales.manager , ...)	Opportunity summary, deal-risk, next-best-action (review), account brief, proposal outline (review)	C o a s
Business Development (business.development.*)	Account/relationship summaries, proposal support	A a
Presales (presales.*)	Proposal draft outline (review)	A e r
Marketing (marketing.*)	Campaign plan (review), email copy (review), audience suggestion (review), performance summary	C a r
Social (social.media.marketing.*)	Caption (review), hashtags, comment sentiment	C
Support (support.*)	Ticket summary, suggested response (review), article recommendation, escalation summary	C r q
Customer Success (customer.success.*)	Onboarding plan (review), health/churn explanation, adoption (review), QBR/EBR (review), renewal strategy (review)	C -
Partner / Reseller Manager (partner.manager , reseller.manager)	Performance summary, action plan (review), inactivity explanation	C r
Training / Knowledge (admin role)	Lesson summary, quiz generator, learning-path	

Login (persona)	AI they apply	
	suggestion; author + publish grounding knowledge	
Executive Leadership (<code>executive.leadership</code>)	Executive insight summaries over dashboards	A
Customer Portal User (<code>customer.portal.user</code>)	Customer-query bot (grounded, cited, auto-escalating)	F
AI Administrator / Super Admin (<code>super.admin</code>)	Govern it all: settings, prompts, agents, review queue	p a

Sign in on the demo (tenant `apar-elite` , password `AparDemo@2026!`) as any persona to see exactly which AI surfaces appear for that role.

Part 5 — How to apply AI in workflows

Workflows can be **driven by AI** and can **invoke AI** — automation plus the governed AI layer.

5.1 AI as a trigger

Two triggers fire on AI-derived signals:

- `ai_score_changed` — e.g. a lead's AI score crosses a threshold.
- `customer_health_changed` — e.g. a CS health score drops.

Use these to react automatically: *"when the AI lead score $\geq$ 80, notify the owner and create a task."*

5.2 AI as an action

Two workflow actions call the AI layer (they reference the registries from Part 3):

Action	What it does	References
<code>run_ai_prompt</code>	Runs a registered prompt template against the record's context.	a Prompt Registry <code>templateKey</code>
<code>run_ai_agent</code>	Invokes a registered agent (with its tools, scope, approval rule).	an Agent Registry <code>agentKey</code>

5.3 Build an AI-powered workflow (step by step)

UI: `/workflows` → New. **API** shown alongside.

1. **Trigger** — choose `ai_score_changed` (module `leads`).
`POST /api/v1/workflows { "name":"Hot-lead AI assist","module":"leads","triggerType":"ai_score_changed" }`
2. **Condition** — only for hot leads: `score gte 80`.
`"conditions": [{"field":"score","operator":"gte","value":80}]`
3. **AI action** — draft a follow-up via a registered prompt: `POST /api/v1/workflows/:id/actions { "actionType":"run_ai_prompt","sequence":1,"actionConfig":{"templateKey":"leads.followup_email"} }`
4. **Follow-on action** — notify the owner: `{ "actionType":"send_notification","sequence":2 }`.
5. **Test** — `POST /api/v1/workflows/:id/run { "context": { "score": 92 } }`, then check `GET /api/v1/workflows/:id/runs`.
6. **Activate** — `PATCH /api/v1/workflows/:id { "status":"active" }`.

Because `leads.followup_email` is a *sensitive* draft, its output is held for **review** — the workflow produces a draft, not an autonomous send, keeping a human in the loop. Agents with `requiresHumanApproval` behave the same way.

Part 6 — Governance & safety (what protects you)

Control	Effect
Human-in-the-loop review	Sensitive action/agent output is <code>pending_review</code> ; approve/reject at <code>/ai-actions (POST /ai/actions/runs/:runId/review)</code> . Only approved output is usable.
Grounding + citations	Knowledge answers cite approved sources and won't fabricate.
Escalation	The customer bot escalates (raises a ticket) instead of guessing; gaps are logged for authors.
Redaction	<code>redactionEnabled</code> strips sensitive data at the gateway.
Rate limiting	<code>rateLimitPerMinute</code> caps usage.
Logging & usage	<code>loggingEnabled</code> records every run; review at <code>/ai/logs</code> , <code>/ai/usage</code> (retention <code>AI_LOG_RETENTION_DAYS</code>).
Permissioned data scope	Agents declare <code>own/team/module/tenant</code> ; retrieval is permission-filtered.
Audit	AI configuration changes and approvals are written to the audit log.

Permissions quick reference

To...	You need
Use Ask AI / run an AI action	<code>ai.use_ai</code> or the module's <code><module>.use_ai</code>
Review/approve a sensitive AI output	<code>ai.approve</code> / <code>ai.manage_ai</code> , or <code><module>.approve</code>
Manage prompts & agents	<code>ai.manage_ai</code> (create/edit), <code>ai.approve</code> (approve prompt versions)
Change tenant AI settings	<code>ai.configure</code> / <code>ai.manage_ai</code>
See AI logs & usage	<code>ai.view</code> , <code>ai.view_dashboard</code> , <code>ai.manage_ai</code> , or <code>ai.configure</code>
Add AI steps to a workflow	<code>workflows.manage_workflow</code> / <code>workflows.configure</code> / <code>workflows.edit</code>

Quick start by goal

- **Turn AI on for the tenant:** Admin → AI settings → set provider/model, enable, set a rate limit, enable redaction + logging.
- **Let a team use AI:** grant that role `*.use_ai` (or `ai.use_ai`) in `/admin/rbac`.
- **Change what the AI says:** edit the template in `/ai-prompts`, approve, activate.
- **Automate with AI:** add a `run_ai_prompt` / `run_ai_agent` action to a workflow (Part 5).
- **Keep it safe:** leave sensitive actions on review; watch `/ai-actions` and `/ai/usage`.

Troubleshooting

Symptom	Cause	Fix
AI buttons missing	Role lacks <code>*.use_ai</code> / <code>ai.use_ai</code> , or <code>isEnabled</code> is off	Grant the permission; enable AI in settings
Output stuck "pending review"	It's a sensitive action	An approver acts in <code>/ai-actions</code>
Bot says it can't find an approved answer	No approved/published knowledge matched	It escalates; author an article for the logged gap
Placeholder-looking AI output	Live provider execution not yet enabled	Add provider keys / enable the execution backend
<code>403</code> calling an <code>/ai/*</code> endpoint	Token's role lacks the permission above	Use an admin/AI-governor token

Companion documents: **Module Guide**, **Persona-Wise Workflow Guide**, and **Admin Configuration Guide**. Deeper design references live in `docs/ai/` (Architecture, Gateway, Governance, Prompt & Agent Registry, RAG, Customer-Query). Source of record: the live `/ai/*`, `/customer-query`, and `/workflows` APIs.